Mastering PHP: From Syntax to Database Applications

A Comprehensive Guide for Computer Science Students

Module 1: Introduction to PHP

1.1 What is PHP?

- **PHP** stands for **Hypertext Preprocessor**.
- It is an **open-source, server-side scripting language**, meaning the code executes on the web server, and the result is sent to the client's browser as plain HTML.
- It is widely used for web development and can be embedded directly into HTML code.

1.2 Why Use PHP?

- **Easy to Learn:** The syntax is heavily inspired by C, C++, and Java, making it highly intuitive for programming students.
- **Platform Independent:** Runs smoothly on Windows, Linux, Unix, and macOS.
- **Server Compatibility:** Compatible with almost all local and cloud servers used today (Apache, Nginx, IIS).
- **Database Support:** Seamlessly connects to a massive range of databases, most notably MySQL.
- **Huge Community:** Being one of the oldest web languages (powering over 75% of the web, including WordPress), it has endless documentation and troubleshooting resources.

1.3 How PHP Works on a Web Server

1. A user requests a URL ending in .php (e.g., index.php).
2. The web server (like Apache) intercepts the request and passes the file to the **PHP Engine**.
3. The PHP Engine processes the instructions, communicates with databases if necessary, and compiles the code.
4. The server returns clean, standard **HTML/CSS** back to the client's web browser. The user never sees the actual PHP source code.

Module 2: Basic Syntax & Sample Program

2.1 PHP Tags and File Extension

- Every PHP file must be saved with a .php extension.
- PHP scripts must be enclosed within standard PHP tags:

```
<?php
// Your PHP code goes here
?>
```

2.2 Case Sensitivity

- **Keywords, classes, and functions** (like if, else, while, echo) are **NOT** case-sensitive. ECHO, Echo, and echo do the same thing.
- **Variable names** are highly **case-sensitive**. $color and $Color are treated as two completely distinct variables.

2.3 First Sample Program

Here is a basic structure mixing HTML and PHP:

```
<html>
<head>
    <title>My First PHP Page</title>
</head>
<body>

<h1>Welcome to My Course</h1>

<?php
// This is a single-line comment
/*
   This is a multi-line comment
   block used for detailed notes.
*/
echo "Hello, World! Welcome to PHP Programming.";
?>

</body>
</html>
```

Module 3: PHP Variables & Scope

3.1 Creating (Declaring) Variables

- In PHP, a variable starts with the **$** sign, followed by the name of the variable.
- PHP is a **loosely typed language**. You do not need to declare data types explicitly before assigning values; PHP automatically detects it based on the assigned value.

```
$txt = "Hello Students"; // String
$x = 5;                  // Integer
$y = 10.5;               // Float
```

3.2 Rules for PHP Variables

1. Must start with a letter or an underscore (_).
2. Cannot start with a number.
3. Can only contain alphanumeric characters and underscores (A-z, 0-9, and _).

3.3 Variable Scope

PHP has three distinct variable scopes:

1. **Local:** A variable declared *inside* a function can only be accessed within that function.
2. **Global:** A variable declared *outside* a function. To access a global variable inside a function, you must use the global keyword.
3. **Static:** When a function finishes execution, its local variables are usually deleted. Use the static keyword if you want a local variable to retain its value for the next time the function is called.

```
$global_var = 10; // Global Scope

function testScope() {
    $local_var = 5; // Local Scope
    global $global_var;
    echo "Global variable inside function: " . $global_var;
}
```

Module 4: PHP Data Types

Variables can store different kinds of data. PHP supports the following primary data types:

Data Type	Description	Example
String	A sequence of characters enclosed in quotes.	"$txt = 'Hello';"
Integer	A non-decimal numerical value between -2,147,483,648 and 2,147,483,647.	"$x = 5985;"
Float (Double)	A number with a decimal point or an exponential form.	"$y = 10.365;"
Boolean	Represents two possible states: true or false.	"$status = true;"
Array	Stores multiple values in a single variable.	"$cars = array('Volvo','BMW');"
Object	An instance of a programmer-defined class.	Explicitly defined via classes.
NULL	A special data type that can have only one value: NULL.	"$z = null;"

Module 5: PHP Operators

Operators are symbols used to perform operations on variables and values.

5.1 Arithmetic Operators

Used for common mathematical operations:

- **+ (Addition), - (Subtraction), * (Multiplication), / (Division), % (Modulus/Remainder), ** (Exponentiation)**

5.2 Assignment Operators

Used with numeric values to write a value to a variable:

- **=, +=, -=, *=, /=, %= (e.g., $x += $y is equivalent to $x = $x + $y)**

5.3 Comparison Operators

Used to compare two values (returns a Boolean):

- **== (Equal value)**
- **=== (Identical: Equal value and equal data type)**
- **!= or <> (Not equal)**
- **>, <, >=, <=**

5.4 Increment / Decrement Operators

- **++$x (Pre-increment), $x++ (Post-increment)**
- **--$x (Pre-decrement), $x-- (Post-decrement)**

5.5 Logical Operators

Used to combine conditional statements:

- **and / && (True if both are true)**
- **or / || (True if at least one is true)**
- **xor (True if either is true, but not both)**
- **! (Not: Reverses the state)**

Module 6: Control Structures (If-Else & Loops)

6.1 Conditional Statements

Conditional statements are used to perform different actions based on different conditions.

- **if Statement:** Executes code if one condition is true.
- **if...else Statement:** Executes code if a condition is true, and another code block if it is false.
- **if...elseif...else Statement:** Checks multiple distinct conditions in sequence.
- **switch Statement:** Selects one of many blocks of code to be executed based on a matching expression.

Syntax Example:

```
$marks = 75;

if ($marks >= 80) {
    echo "Grade: Distinction";
} elseif ($marks >= 60) {
    echo "Grade: First Class";
} else {
    echo "Grade: Pass/Fail";
}
```

6.2 Loops

Loops execute the same block of code repeatedly as long as a specified condition remains true.

- **while**: Loops through a block of code as long as the specified condition is true.
- **do...while**: Executes the code block once, and then repeats the loop as long as the

condition is true (guaranteed to run at least once).
- **for**: Loops through a block of code a specified number of times.
- **foreach**: Loops through a block of code for each element inside an **Array**.

Syntax Example (foreach):

```
$students = array("Alice", "Bob", "Charlie");

foreach ($students as $name) {
    echo "Student Name: $name <br>";
}
```

Module 7: Database Connection & CRUD Operations

In production, PHP interacts with a relational database system using either the **MySQLi extension** (procedural/object-oriented) or **PDO (PHP Data Objects)**. We will use the Object-Oriented **MySQLi** approach below.

7.1 Connecting to MySQL Database

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "college_db";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Database Connection failed: " . $conn->connect_error);
}
echo "Connected successfully to the database!";
?>
```

7.2 The CRUD Concept

CRUD represents the four foundational operations managing data inside a database storage layer:

1. **C**reate (SQL INSERT)
2. **R**ead (SQL SELECT)
3. **U**pdate (SQL UPDATE)
4. **D**elete (SQL DELETE)

1. CREATE (Insert Data)

```
$sql = "INSERT INTO Students (firstname, lastname, email) VALUES
('John', 'Doe', 'john@example.com')";
```

```
if ($conn->query($sql) === TRUE) {
    echo "New student record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}
```

2. READ (Select & Display Data)

```
$sql = "SELECT id, firstname, lastname FROM Students";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // Output data of each row
    while($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"]. " - Name: " . $row["firstname"]. " "
. $row["lastname"]. "<br>";
    }
} else {
    echo "0 results found.";
}
```

3. UPDATE (Modify Existing Data)

```
$sql = "UPDATE Students SET lastname='Smith' WHERE id=1";

if ($conn->query($sql) === TRUE) {
    echo "Record updated successfully";
} else {
    echo "Error updating record: " . $conn->error;
}
```

4. DELETE (Remove Data)

```
$sql = "DELETE FROM Students WHERE id=1";

if ($conn->query($sql) === TRUE) {
    echo "Record deleted successfully";
} else {
    echo "Error deleting record: " . $conn->error;
}
```

Module 8: 50 Sample Practical Programs

This repository of programs is divided cleanly from basic scripts to advanced database logic. Copy, execute, and analyze these to build muscle memory.

Category A: Basic Syntax & Arithmetic (Programs 1-10)

1. Hello World

```
<?php
echo "Hello World!";
?>
```

2. Simple Addition of Two Numbers

```
<?php
$a = 15;
$b = 25;
$sum = $a + $b;
echo "The sum is: " . $sum;
?>
```

3. Swap Two Numbers Using a Third Variable

```
<?php
$a = 5;
$b = 10;
$temp = $a;
$a = $b;
$b = $temp;
echo "After swapping: a = $a, b = $b";
?>
```

4. Swap Two Numbers Without a Third Variable

```
<?php
$a = 10;
$b = 5;
$a = $a + $b; // 15
$b = $a - $b; // 10
$a = $a - $b; // 5
echo "After swapping: a = $a, b = $b";
?>
```

5. Calculate Area of a Rectangle

```
<?php
$length = 20;
$width = 10;
$area = $length * $width;
echo "Area of Rectangle: " . $area;
?>
```

6. Calculate Area of a Circle

```
<?php
define("PI", 3.14159);
$radius = 7;
$area = PI * $radius * $radius;
echo "Area of Circle: " . $area;
?>
```

7. Convert Celsius to Fahrenheit

```
<?php
$celsius = 32;
$fahrenheit = ($celsius * 9/5) + 32;
echo "$celsius°C is equal to $fahrenheit°F";
?>
```

8. Find Length of a String

```
<?php
$str = "Computer Science";
echo "The length of the string is: " . strlen($str);
?>
```

9. Count Number of Words inside a String

```
<?php
$str = "Welcome to PHP web programming basics.";
echo "Word count: " . str_word_count($str);
?>
```

10. Reverse a String

```
<?php
$str = "Surat";
echo "Reversed string: " . strrev($str);
?>
```

Category B: Conditional Logic & If-Else (Programs 11-20)

11. Check if a Number is Even or Odd

```
<?php
$num = 44;
if($num % 2 == 0) {
   echo "$num is Even";
} else {
   echo "$num is Odd";
}
?>
```

12. Check if a Person is Eligible to Vote

```
<?php
$age = 19;
if($age >= 18) {
   echo "Eligible to vote.";
} else {
   echo "Not eligible to vote.";
}
?>
```

13. Find the Largest of Two Numbers

```
<?php
$x = 45;
$y = 90;
if($x > $y) {
   echo "$x is greater than $y";
} else {
   echo "$y is greater than $x";
}
?>
```

14. Find the Largest of Three Numbers

```
<?php
$a = 10; $b = 25; $c = 15;
if($a >= $b && $a >= $c) {
   echo "$a is the largest number.";
} elseif($b >= $a && $b >= $c) {
   echo "$b is the largest number.";
} else {
   echo "$c is the largest number.";
}
?>
```

15. Check if a Year is a Leap Year

```
<?php
$year = 2024;
if(($year % 400 == 0) || ($year % 4 == 0 && $year % 100 != 0)) {
   echo "$year is a Leap Year.";
} else {
   echo "$year is not a Leap Year.";
}
?>
```

16. Check if a Number is Positive, Negative, or Zero

```
<?php
$num = -7;
if($num > 0) {
   echo "$num is Positive";
} elseif($num < 0) {
   echo "$num is Negative";
} else {
   echo "The number is Zero";
}
?>
```

17. Simple Calculator Using Switch Case

```
<?php
$num1 = 12;
$num2 = 4;
$operator = '/';
```

```
switch($operator) {
   case '+': echo $num1 + $num2; break;
   case '-': echo $num1 - $num2; break;
   case '*': echo $num1 * $num2; break;
   case '/': echo $num1 / $num2; break;
   default: echo "Invalid Operator";
}
?>
```

18. Check if a Character is a Vowel or Consonant

```
<?php
$char = 'e';
switch(strtolower($char)) {
   case 'a': case 'e': case 'i': case 'o': case 'u':
      echo "$char is a Vowel.";
      break;
   default:
      echo "$char is a Consonant.";
}
?>
```

19. Student Grading Scheme

```
<?php
$score = 85;
if($score >= 90) echo "Grade A";
elseif($score >= 75) echo "Grade B";
elseif($score >= 50) echo "Grade C";
else echo "Fail";
?>
```

20. Check if a Number Lies Within a Specific Range (10 to 50)

```
<?php
$num = 27;
if($num >= 10 && $num <= 50) {
   echo "$num is within the range [10, 50]";
} else {
   echo "$num is out of range";
}
?>
```

Category C: Iteration & Loops (Programs 21-30)

21. Print 1 to 10 Using a For Loop

```
<?php
for($i = 1; $i <= 10; $i++) {
   echo $i . " ";
}
?>
```

22. Print 10 to 1 in Reverse Using a While Loop

```
<?php
$i = 10;
while($i >= 1) {
   echo $i . " ";
   $i--;
}
?>
```

23. Print the Multiplication Table of 5

```
<?php
$table = 5;
for($i = 1; $i <= 10; $i++) {
   $res = $table * $i;
   echo "$table x $i = $res <br>";
}
?>
```

24. Calculate Factorial of a Number

```
<?php
$num = 5;
$factorial = 1;
for($i = 1; $i <= $num; $i++) {
   $factorial *= $i;
}
echo "Factorial of $num is: " . $factorial;
?>
```

25. Fibonacci Series up to 10 Terms

```
<?php
$n = 10;
$n1 = 0; $n2 = 1;
echo "Fibonacci Series: " . $n1 . " " . $n2 . " ";
for($i = 3; $i <= $n; $i++) {
   $n3 = $n1 + $n2;
   echo $n3 . " ";
   $n1 = $n2;
   $n2 = $n3;
}
?>
```

26. Sum of First 100 Natural Numbers

```
<?php
$sum = 0;
for($i = 1; $i <= 100; $i++) {
   $sum += $i;
}
echo "The sum is: " . $sum;
?>
```

27. Display All Even Numbers Between 1 and 20

```
<?php
for($i = 1; $i <= 20; $i++) {
   if($i % 2 == 0) {
      echo $i . " ";
   }
}
?>
```

28. Count Digits of an Integer

```
<?php
$num = 14526;
$count = 0;
while($num != 0) {
   $num = (int)($num / 10);
```

```
    $count++;
}
echo "Total digits: " . $count;
?>
```

29. Sum of Digits of a Number

```
<?php
$num = 1234;
$sum = 0;
while($num > 0) {
    $rem = $num % 10;
    $sum += $rem;
    $num = (int)($num / 10);
}
echo "Sum of digits: " . $sum;
?>
```

30. Simple Nested Loop Star Pattern

```
<?php
for($i = 1; $i <= 5; $i++) {
    for($j = 1; $j <= $i; $j++) {
        echo "* ";
    }
    echo "<br>";
}
?>
```

Category D: Arrays & Intermediate Core Concepts (Programs 31-40)

31. Create and Display an Indexed Array

```
<?php
$languages = array("PHP", "Java", "Python", "C++");
foreach($languages as $val) {
    echo $val . "<br>";
}
?>
```

32. Find the Largest Number inside an Array

```
<?php
$numbers = array(14, 56, 22, 89, 43);
$max = $numbers[0];
for($i = 1; $i < count($numbers); $i++) {
   if($numbers[$i] > $max) {
      $max = $numbers[$i];
   }
}
echo "Largest element: " . $max;
?>
```

33. Associative Array Iteration

```
<?php
$ages = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
foreach($ages as $name => $age) {
   echo "Key: " . $name . ", Value: " . $age . "<br>";
}
?>
```

34. Sort Array in Ascending Order

```
<?php
$data = array(5, 1, 4, 2, 8);
sort($data);
print_r($data);
?>
```

35. Search an Element inside an Array (Linear Search)

```
<?php
$fruits = array("Apple", "Banana", "Cherry");
$search = "Banana";
if(in_array($search, $fruits)) {
   echo "$search was found in the array!";
} else {
   echo "$search not found.";
}
?>
```

36. Merge Two Arrays

```
<?php
$arr1 = array("red", "green");
$arr2 = array("blue", "yellow");
$result = array_merge($arr1, $arr2);
print_r($result);
?>
```

37. Basic Function with Arguments

```
<?php
function greetStudent($name) {
   echo "Welcome back, $name!<br>";
}
greetStudent("Ankit");
?>
```

38. Function Returning a Value

```
<?php
function multiply($x, $y) {
   return $x * $y;
}
echo "Result: " . multiply(6, 7);
?>
```

39. Check if a String is a Palindrome

```
<?php
$word = "radar";
if($word == strrev($word)) {
   echo "$word is a palindrome.";
} else {
   echo "$word is not a palindrome.";
}
?>
```

40. Determine Prime Status of a Number

```
<?php
$num = 17;
$isPrime = true;
for($i = 2; $i <= sqrt($num); $i++) {
   if($num % $i == 0) {
      $isPrime = false;
      break;
   }
}
echo $isPrime ? "$num is Prime" : "$num is Not Prime";
?>
```

Category E: Forms & Database CRUD Integration (Programs 41-50)

41. Process HTML Form Data Securely via $_POST

```
// Form HTML processing block
if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $username = htmlspecialchars($_POST['uname']);
   echo "Form received user variable: " . $username;
}
```

42. Complete Database Connection Verification Script

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
if ($conn->connect_error) {
   die("Error connecting: " . $conn->connect_error);
}
echo "Database configuration running clean.";
?>
```

43. Create a Database Table via PHP Execution

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$sql = "CREATE TABLE Users (id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(30) NOT NULL)";
if ($conn->query($sql) === TRUE) echo "Table Users created successfully";
?>
```

44. CRUD - INSERT execution script

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$sql = "INSERT INTO Users (username) VALUES ('Aadarsh')";
if($conn->query($sql) === TRUE) echo "Data inserted successfully";
?>
```

45. CRUD - SELECT and Display Output Rows

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$result = $conn->query("SELECT * FROM Users");
while($row = $result->fetch_assoc()) {
   echo "User ID: " . $row['id'] . " Name: " . $row['username'] . "<br>";
}
?>
```

46. CRUD - UPDATE record based on conditional ID

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$sql = "UPDATE Users SET username='Mukesh' WHERE id=1";
if($conn->query($sql) === TRUE) echo "Record altered successfully.";
?>
```

47. CRUD - DELETE statement query execution

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$sql = "DELETE FROM Users WHERE id=1";
if($conn->query($sql) === TRUE) echo "Record purged.";
?>
```

48. Count Rows inside a Table Grid

```
<?php
$conn = new mysqli("localhost", "root", "", "test_db");
$result = $conn->query("SELECT id FROM Users");
echo "Total Rows: " . $result->num_rows;
?>
```

49. Set and Read a Cookie Variable

```
<?php
setcookie("user", "Kanika", time() + (86400 * 30), "/");
if(isset($_COOKIE["user"])) {
   echo "Cookie value: " . $_COOKIE["user"];
}
?>
```

50. Initialize, Assign, and Destroy a Session Variable

```
<?php
session_start();
$_SESSION["role"] = "Administrator";
echo "Session started. Role assigned: " . $_SESSION["role"];
// To clear tracking completely use: session_destroy();
?>
```